EvoCode-Bench: Evaluating Coding Agents in Multi-Turn Iterative Interactions

Anonymous Author(s)

Abstract

Coding agents are increasingly used as iterative development partners, but most benchmarks still evaluate one specification followed by one final assessment. This misses whether agents can maintain executable artifacts as requirements evolve. We introduce EvoCode-Bench, a benchmark of 26 stateful coding tasks spanning 227 evaluated rounds. Each task keeps the agent's workspace across 5–15 rounds, expresses requirements through observable behavior, and uses cumulative executable tests to check both new requirements and regressions against still-active prior ones. We evaluate 13 coding agents with two complementary metrics: MT@4, a four-attempt fail-stop multi-round score, and SR, a single-round score from a reference-completed prior state. For most agents, SR exceeds MT@4 by 22–40 points, indicating that isolated instruction-following does not imply persistent reliability. The gap also changes model rankings: the highest-SR agent (78.9) ranks only third in persistent execution (44.0 MT@4). Even the strongest agents earn only about half of the multi-round credit, and aggregate pass rate drops below half of round-1 performance by round 5. Failure analysis shows tier-dependent behavior: weaker agents fail early, while stronger agents survive long enough to expose specification-tracking and regression failures. These results show that single-round and persistent multi-turn coding evaluate different capability dimensions. We release the benchmark data and Harbor multi-turn infrastructure.[1]

1 Introduction

Coding agents have evolved from code-completion tools into systems that plan, edit files, execute commands, and interact with development environments. Products such as Claude Code [1], Cursor [2], Codex [23], and Windsurf [30] are now used for tasks such as debugging, data analysis, service deployment, and infrastructure management. In these settings, the agent must operate within a persistent workspace: each edit changes files, dependencies, interfaces, and tests that later interactions inherit. This makes coding a useful setting for evaluating tool-using agents because decisions leave executable traces and correctness can be checked through behavior.

Current evaluation only partly matches this setting. Function-level benchmarks [5, 3, 13, 42] evaluate localized programming ability through one-shot generation. Repository-level benchmarks test repair, completion, and feature addition in realistic codebases [19, 14, 34, 18, 10, 7]. SWE-Gym [24] turns software engineering tasks into training environments. Environment-level benchmarks such as AppWorld [27] and Terminal-Bench [22] evaluate agents that interact with executable environments. Multi-turn evaluation has also been studied in dialogue, tool use, and general agent settings [41, 15, 16, 35, 21]. These efforts make coding evaluation more realistic, but their usual evaluation unit is still one task specification followed by one terminal assessment. Persistent coding raises a different problem: later turns must build on the agent's own artifact while preserving all requirements that remain active.

[1] Benchmark data: `https://huggingface.co/datasets/anonymousee8/evocodebench`; Harbor multi-turn: `https://huggingface.co/anonymousee8/harbor_multiturn`.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

In practice, users interact with coding agents iteratively: they add output formats, correct behavior, introduce constraints, and supersede earlier requirements. Unlike dialogue, where an earlier poor response can often be corrected in text, coding materializes decisions into file layouts, schemas, APIs, and dependency choices. These commitments constrain later work and can create regressions. A benchmark for this setting must therefore include cross-round dependencies, cumulative executable tests, and scoring that identifies when the evolving workspace first stops satisfying the active specification.

To address this gap, we introduce EVOCODE-BENCH, to our knowledge the first benchmark specifically designed to evaluate coding agents in interactive, persistent multi-turn sessions where requirements naturally evolve and conflict. EVOCODE-BENCH contains 26 multi-round tasks and 227 evaluated rounds, with 5 to 15 rounds per task. The benchmark is organized along two orthogonal dimensions: *Interaction style*, which captures how users communicate across rounds, and *Engineering activity*, which captures whether each round demands incremental construction, specification conflict, quality-driven review, or legacy migration. Since correct agents may build different internal designs, EVOCODE-BENCH evaluates behavior rather than implementation paths: instructions state observable requirements, and verification scripts exercise those requirements through execution instead of inspecting code structure. Section 3 presents the full taxonomy and task construction pipeline.

We systematically evaluate 13 leading coding agents using the Terminus-2 harness [12] and Harbor execution protocol [11]. We extend Harbor so that one Docker workspace and one agent session persist across round boundaries while cumulative verifiers are swapped and state lineage is recorded. The results show three findings. First, SR exceeds MT@4 by 22 to 40 points for most agents, and the gap reranks models: Opus-4.6 has the highest SR (78.9) but only the third-highest MT@4 (44.0). Second, EVOCODE-BENCH is challenging even for the strongest models: only two agents exceed half of the multi-round credit, and top agents still fail on many long trajectories. Third, performance drops quickly with depth, and failure analysis separates early missed requirements in lower-tier agents from conflict, correction, and regression failures that emerge among stronger agents. These results show that preserving correctness across evolving requirements remains an unsolved capability boundary.

Contributions. We contribute: (i) a task definition for interactive persistent multi-turn coding with evolving and conflicting requirements; (ii) EVOCODE-BENCH, with 26 tasks, 227 evaluated rounds, persistent workspaces, reference deltas, cumulative verifiers, fail-stop scoring, and a two-axis taxonomy; (iii) open-source Harbor multi-turn extensions for continuous sessions, verifier swaps, reference fast-forwarding, snapshot/resume lineage, and fail-stop accounting; and (iv) a 13-agent evaluation showing that single-round performance substantially overstates persistent reliability and changes model rankings.

2 Related Work

Code Generation and Repair Benchmarks Code evaluation benchmarks have progressed from function-level generation [5, 3] through instruction-following with library calls [13, 42] to repository-level task solving. SWE-bench [14] and SWE-bench Multimodal [34] require agents to resolve real issues; RepoBench [19] evaluates repository-level completion. A parallel line studies repository-level generation: EvoCodeBench [17], DevEval [18], RepoExec [10], CodeS [36], NL2Repo-Bench [7], and NoCode-bench [6] each advance different facets of repository construction and feature implementation. SWE-Gym [24] turns software engineering tasks into training environments. Two recent efforts add sequential structure: SWE-EVO [38] requires implementing an entire software release under cumulative regression tests from a single specification, and SlopCodeBench [9] evaluates code quality across sequential additive checkpoints. Both test sustained correctness within a single session but use fixed or additive specifications without interactive user dialogue or requirement revision.

EVOCODE-BENCH differs in its interactive multi-turn design: requirements evolve and conflict across rounds through simulated user dialogue, and the cumulative verifier checks the agent's own accumulated workspace at every round. This tests regression avoidance, conflict resolution, and incremental adaptation under requirement change that fixed-specification benchmarks do not measure.

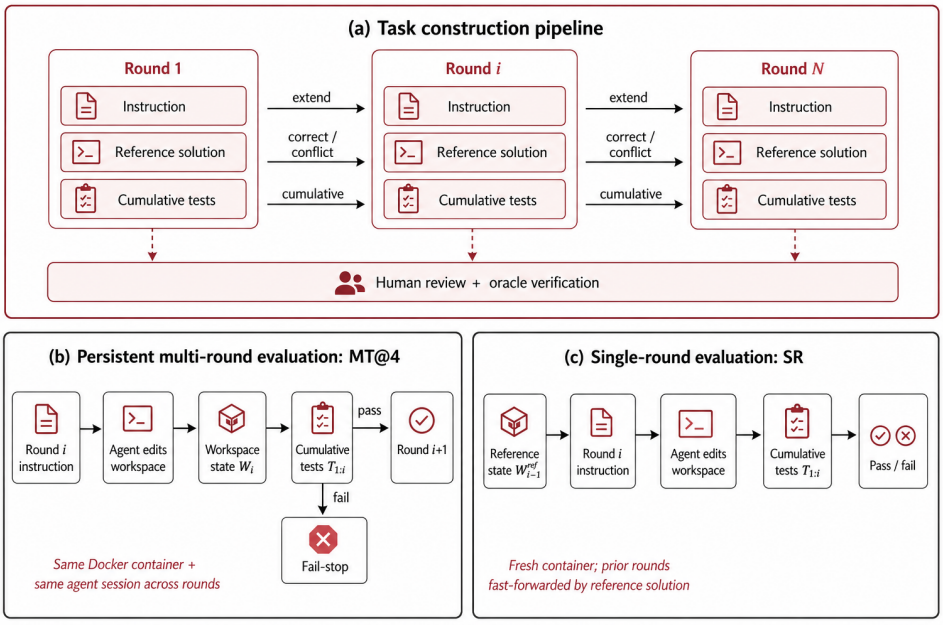

Figure 1: Overview of EvoCode-Bench. (a) Each round contains an instruction, reference solution, and cumulative tests checked by human review and oracle verification. (b) MT@4 keeps one Docker environment and agent session across rounds, with fail-stop termination. (c) SR fast-forwards to the reference state before each target round.

Autonomous Coding Agent Evaluation Recent agent work studies systems that plan, use shell commands, and interact with development environments. SWE-agent [33] and OpenHands [29] provide agent-computer interfaces; Terminal-Bench [22] and AppWorld [27] evaluate executable-environment interaction. AutoCodeRover [37], Agentless [31], and SWE-Dev [8] target repository repair or feature development. These settings still usually define success from one task specification and one trajectory. Our work introduces iterative interaction into this loop. Built on Harbor [11], EvoCode-Bench supports persistent environments, cumulative tests, and state accounting across rounds.

Multi-Turn Interaction Evaluation Multi-turn evaluation spans dialogue, general agents, and tool use. MT-Bench [41], MT-Eval [15], and LMSYS-Chat-1M [40] study conversational interaction; Laban et al. [16] show that distributing information across turns can sharply reduce performance. AgentBench [20], OSWorld [32], BFCL [25], τ-bench [35], and ToolSandbox [21] evaluate sequential decision making or stateful tool use. Coding-specific work studies multi-turn generation, dependency-ordered code flow, and security [39, 28, 26]. EvoCode-Bench is complementary: it evaluates repository-level development chains by executing cumulative tests after every round, so each intermediate artifact is behaviorally verified.

3 EvoCode-Bench

This section defines EvoCode-Bench as a benchmark for persistent multi-turn coding. The central design choice is to treat coding as an evolving workspace rather than isolated prompts: each response changes files, dependencies, interfaces, and tests that later rounds inherit. EvoCode-Bench therefore keeps the same workspace and agent session across rounds, carries active requirements forward through cumulative tests, and records the first round where the accumulated implementation violates the active specification.

Table 1: Task taxonomy and distribution. Each count cell reports *tasks / rounds*, so the table reflects both the multi-round task inventory and the round-level evaluation units used for single-round analysis. Expl., Contr., and Doc. denote explorative, contractual, and document-driven interaction styles.

Engineering Activity	Capability measured	Expl.	Contr.	Doc.	Total
Construction	Building a system incrementally while preserving earlier features and interfaces.	9 / 80	3 / 37	1 / 7	13 / 124
Spec Evolution	Updating an implementation after a later round overturns a core assumption.	1 / 8	1 / 7	1 / 7	3 / 22
Review	Improving non-functional properties such as performance, security, and observability without regression.	3 / 21	1 / 7	1 / 9	5 / 37
Migration	Moving a legacy system to a new implementation paradigm while keeping backward compatibility.	3 / 29	1 / 7	1 / 8	5 / 44
Total		**16 / 138**	**6 / 58**	**4 / 31**	**26 / 227**

3.1 Task Definition and Taxonomy

A EVOCODE-BENCH task has N rounds executed in one persistent Docker container (Figure 1b). At round i, the agent receives instruction $\mathcal{I}_i$ and edits workspace $\mathcal{W}_{i-1}$ into $\mathcal{W}_i$. The verifier $\mathcal{T}_{1:i}$ tests all active requirements through round i; the next instruction is issued only after a pass. Each round yields $r_i \in \{0, 1\}$. If $r_i = 0$, evaluation stops and later rounds receive zero credit.

Fail-stop scoring makes the failure point interpretable. Continuing after failure would mix recovery from an invalid workspace with preservation of a valid evolving state. EVOCODE-BENCH measures the latter in its main multi-turn score; Appendix F.5 discusses recovery-oriented and reference-fast-forwarded alternatives.

Two-dimensional taxonomy. Multi-turn coding tasks differ in both communication style and engineering pressure. EVOCODE-BENCH therefore labels each task along two axes: interaction style and engineering activity.

Interaction styles. The first axis captures where requirements live. **Explorative** tasks start with a detailed request and then use terse follow-ups, testing intent recovery from prior conversation and repository state. **Contractual** tasks provide detailed behavioral specifications at every round, including revisions to earlier behavior. **Document-driven** tasks place persistent semantics in project artifacts such as specifications or `AGENTS.md`, testing whether agents treat repository documents as part of the active specification.

Engineering activities. The second axis captures the dominant engineering pressure: construction, specification evolution, review-driven improvement, or migration with compatibility preservation. Table 1 gives definitions and distribution. Later tables abbreviate these labels as Con, Spec, Rev, Mig, Exp, Ctr, and Doc.

Reporting both axes supports task-level multi-turn analysis and round-level single-round analysis; Appendix C gives a released example.

3.2 Data Collection Pipeline

Constructing multi-turn tasks requires coherent requirement chains, cumulative behavioral tests, and incremental reference solutions (Figure 1a). A valid task must make later rounds depend on earlier state while still allowing multiple correct implementation paths. We use a four-stage quality pipeline; Appendices B and D give the released format and additional QA details.

Task design protocol. The main validity risk is that agents may take different early implementation paths. Instructions therefore describe observable behavior, tests verify execution rather than source layout, and each round's tests are independent of prior reference patches. Each task begins with a substantial system foundation and includes at least one correction and one conflict, so agents must both extend and revise existing behavior.

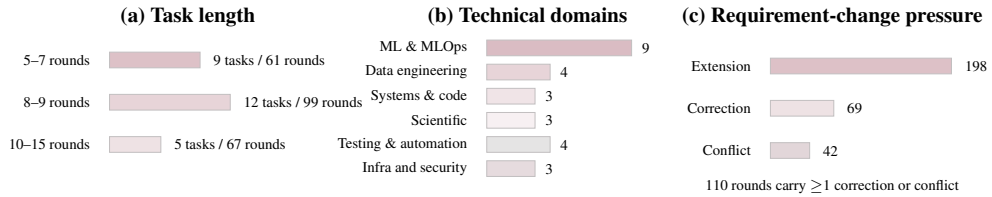

Figure 2: Dataset statistics for EVOCODE-BENCH beyond the taxonomy distribution in Table 1. The benchmark contains 26 multi-turn tasks and 227 evaluated rounds, with 5 to 15 rounds per task, broad technical coverage, and frequent requirement corrections or conflicts.

Task construction and internal review. Engineers author complete chains, including instructions, reference solutions, tests, and Docker environments. LLM assistance is used for drafting candidate task ideas and failure-analysis prompts, but released instructions, tests, reference deltas, and accepted diagnostic labels are author-reviewed. Two independent reviewers then check instruction coherence and three-way alignment among instructions, tests, and solutions. Reviewers ask whether a competent implementation that differs from the reference can pass, whether each test assertion follows from a current requirement, and whether later rounds preserve all requirements that have not been explicitly superseded. Tasks with critical findings are revised before oracle verification.

Oracle verification. Each reference solution is executed in the full environment and must obtain $r_i = 1$ for every round. This catches incorrect solutions, missing dependencies, nondeterministic tests, and unsatisfiable specifications.

Cross-validation. Final cross-validation verifies reference behavior, traces every assertion to a current specification, and audits shortcut strategies. We inspect whether a model could pass by hard-coding fixture outputs, ignoring documented formats, or satisfying only the latest round while breaking earlier requirements. Unresolved tasks are revised or removed. The resulting benchmark is intentionally smaller than broad single-turn collections, but each released task supplies a longer and more controlled development trajectory.

3.3 Dataset Statistics

EVOCODE-BENCH contains 26 tasks and 227 evaluated rounds. Figure 2 summarizes scale, task length, technical coverage, and requirement-change pressure; Table 1 gives the taxonomy cross product. Each task is long enough for early implementation choices to constrain later work, while short enough for failures to be traced to concrete requirement changes.

The benchmark covers MLOps, data engineering, systems programming, scientific computing, testing, automation, cloud/devops, security, and compiler implementation. We intentionally avoid treating domain labels as the primary taxonomy because technical domain and multi-turn behavior are not the same property: an MLOps task can be construction-oriented, review-driven, or migration-oriented depending on the requirement chain. Domain coverage instead supports external validity, while the two-dimensional taxonomy supports analysis of multi-turn behavior.

Rounds are annotated with non-exclusive change types: *extension*, *correction*, and *conflict*. The distinction captures stale behavior after specification changes, not only missing latest-round features.

Instruction length follows interaction style: explorative tasks shorten after the first round, contractual tasks stay detailed, and document-driven tasks move semantics into artifacts such as `AGENTS.md`. All styles use the same executable verifier.

3.4 Evaluation Protocol

All evaluations use Harbor with the Terminus-2 agent harness, which gives each model shell access inside a Docker container. In multi-round evaluation, the same container and agent session persist across rounds. Harbor handles round sequencing, cumulative verification, snapshotting, SR reference fast-forwarding, and fail-stop accounting, turning a single-instruction trial into a round-indexed trial with continuous workspace, session, reward log, and resume lineage. We use the same harness,

environment, tool interface, and prompt protocol for all models; Appendix F.1 gives the exact configuration, and Appendix F.3 details the Harbor multi-turn extensions.

For task t with N_t rounds, let $r_{t,a,i} \in \{0, 1\}$ be the cumulative verifier reward for attempt a at round i. Under fail-stop, if attempt a first fails at round i, then $r_{t,a,j} = 0$ for every $j > i$. The single-attempt multi-turn score is

$$S_{t,a} = \frac{1}{N_t} \sum_{i=1}^{N_t} r_{t,a,i}. \tag{1}$$

We report **MT@4**, defined as $\frac{1}{|\mathcal{D}|} \sum_t \frac{1}{N_t} \sum_i \max_{a \le 4} r_{t,a,i}$. Equivalently, MT@4 gives credit for a round if any of four attempts reaches that round with a workspace that still satisfies the cumulative verifier. **SR** is $\frac{1}{\sum_t N_t} \sum_{t,i} s_{t,i}$, where $s_{t,i}$ is the binary reward for solving round i after Harbor fast-forwards earlier rounds with reference deltas. SR measures a different condition from MT@4: whether the agent can solve a target round when all earlier work has been completed by the reference solution. A large MT@4–SR gap therefore means that isolated instruction-following ability is not sufficient, under this protocol, to maintain the agent's own long-horizon workspace.

Comp is $\frac{1}{|\mathcal{D}|} \sum_t \mathbf{1}[\max_{a \le 4} r_{t,a,N_t} = 1]$, the fraction of tasks completed through the final round in at least one attempt. We also report **Avg. Turns** and **Output Tok. (K)** for multi-round runs, with partial trajectories scaled to the full task horizon. Each $\mathcal{T}_{1:i}$ tests all still-valid requirements through round i, enabling regression detection. Appendix F gives exact metric accounting.

4 Experiments

4.1 Evaluation Setup

We evaluate the 13 coding agents in Table 2 under a shared protocol. Each agent-task pair runs in a dedicated Docker container, and correctness is measured only by executing the task verifier. Multi-round evaluation keeps the same workspace and agent session across rounds and uses four independent attempts. Single-round evaluation fast-forwards the workspace to the reference state before the target round. Appendix F.2 maps compact model labels to full names and endpoint identifiers.

EvoCode-Bench contains 26 tasks and 227 evaluated rounds, with 5 to 15 rounds per task. We aggregate by agent, engineering activity, interaction style, and round index to distinguish single-round performance, persistent execution, and degradation over longer requirement chains.

We report **MT@4**, **SR**, **Comp**, **Avg. Turns**, and **Output Tok. (K)** as defined in Section 3.4, and sort the main table by MT@4. MT@4 asks whether any of four attempts can maintain a valid evolving workspace; SR asks whether a single attempt can solve the same round from a reference-completed state. Together they separate accumulated-state reliability from isolated instruction-following ability.

4.2 Main Results

The benchmark is challenging even for the strongest agents. Opus-4.7 leads with 54.0 MT@4 and GPT-5.5 follows at 52.4; only these two agents exceed half of the total multi-round credit. Opus-4.6 reaches 44.0, and every other agent scores at or below 36.2. Full-task completion rates are correspondingly low: Opus-4.7 completes 42.3% of tasks through the final round, GPT-5.5 38.5%, and no agent below the top three exceeds 23.1%.

Multi-turn execution amplifies the gap across capability tiers. Grouping agents into top, middle, and lower tiers by MT@4, the top-to-lower ratio is 5.9× (50.1 vs. 8.4), compared with only 2.3× under SR (76.7 vs. 33.1). Thus persistent execution spreads agents much further apart than single-round evaluation. All agents except MiMo-V2.5-Pro have SR substantially above MT@4, with gaps of 21.9–39.5 points; Opus-4.6 is the clearest reranking, with the highest SR (78.9) but only the third-highest MT@4 (44.0).

The gap is not an artifact of giving MT@4 four attempts and SR one attempt. At round 1, the best-of-four advantage makes MT@4 exceed SR; from round 2 onward, the ordering reverses (Appendix G.8). The reversal indicates that accumulated state dominates the attempt-count advantage

Table 2: Main results on EvoCode-Bench. Rows are sorted by **MT@4**. **MT@4** is the four-attempt fail-stop multi-round score, **SR** is reference-fast-forward single-round pass rate, and **Comp** is full-task completion. **Avg. Turns** is agent–model exchanges per full task; **Output Tok. (K)** is generated-token usage in thousands. Activity columns abbreviate construction, specification evolution, review-driven improvement, and migration; style columns abbreviate explorative, contractual, and document-driven tasks.

	Overall					Engineering Activity				Interaction Style		
Agent	**MT@4**	**SR**	**Comp**	**Avg. Turns**	**Output Tok. (K)**	**Con**	**Spec**	**Rev**	**Mig**	**Exp**	**Ctr**	**Doc**
Opus-4.7	**54.0**	76.7	**42.3**	590.6	50.0	**58.8**	42.9	**70.0**	32.3	41.8	**64.1**	82.1
GPT-5.5	52.4	74.4	38.5	456.3	74.1	57.3	33.3	50.0	**53.6**	**51.0**	42.6	75.0
Opus-4.6	44.0	**78.9**	34.6	747.5	734.2	37.4	**66.7**	44.0	47.9	33.1	35.5	**100.0**
GLM-5.1	36.2	63.9	15.4	859.8	104.2	29.6	61.9	43.1	30.9	23.3	30.5	94.4
Kimi-K2.6	31.9	59.0	23.1	1155.5	92.5	29.5	33.3	42.5	26.9	21.6	29.5	75.0
DS-V4-Pro	30.6	56.4	19.2	1134.8	168.8	37.3	0.0	33.0	29.4	23.7	20.1	75.0
Qwen3.6-Plus	29.4	57.3	15.4	629.3	103.1	30.2	45.8	22.5	24.3	28.6	19.3	50.0
MiMo-V2.5	17.3	7.9	11.5	754.8	125.7	20.6	0.0	22.5	13.6	17.6	12.1	25.0
Gemini-3.1	13.7	46.7	11.5	261.3	72.7	12.0	0.0	20.0	20.0	10.4	0.0	50.0
DS-V4-Flash	9.4	46.3	0.0	1104.7	148.7	5.2	0.0	28.6	6.9	9.4	7.1	13.9
Qwen3.5-397B	4.6	44.1	0.0	587.8	53.0	5.4	0.0	2.2	7.9	4.4	6.1	2.8
MiniMax-M2.7	3.7	30.0	0.0	600.4	59.2	2.2	0.0	6.7	6.9	3.3	2.0	8.3
Doubao-2.0	1.9	23.8	0.0	211.1	18.5	2.9	0.0	0.0	2.5	3.3	0.0	0.0

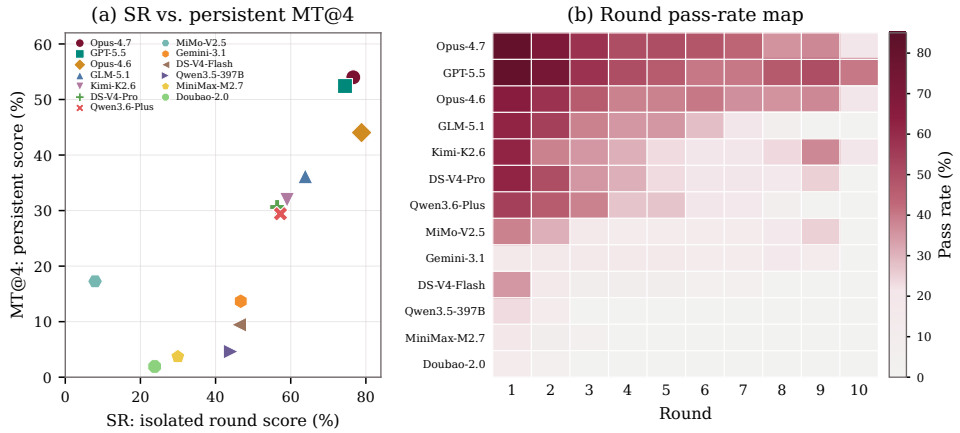

Figure 3: (a) SR vs. persistent MT@4 for each agent. (b) Per-round pass rate heatmap; rows are agents sorted by MT@4, columns are rounds.

after the first round. MiMo-V2.5-Pro is the sole exception (SR 7.9 vs. MT@4 17.3), largely because 60% of its multi-turn failures occur at round 1. This case illustrates that SR rewards extending a reference-completed codebase, whereas MT@4 rewards building and maintaining one's own.

Performance degrades sharply across rounds. Round-level MT@4 pass rates fall from 46.7 at round 1 to 26.9 at round 3, 21.3 at round 5, and 7.7 at round 10. Some decline reflects the smaller set of long tasks, but the within-task heatmap in Figure 3(b) also shows falling pass rates for most agents. SR further separates round difficulty from persistent-state degradation: it remains stable at 52–57% for rounds 3–8 while MT@4 continues to decline (Figure 4). The SR–MT@4 gap reaches 33 points by round 5 and 41 points by round 8, indicating that accumulated workspace state drives much of the multi-turn drop. A controlled comparison reinforces this: 57.0% of failed MT@4 round records are solvable under SR from a reference-completed state (Appendix G.11). The penalty grows with depth: only 15.0% of round-1 MT failures are SR-solvable, rising to 59.0% at round 7 and above 80% beyond round 12 (Figure 9a).

Cross-attempt variance increases with round depth. MT@4 credits a round if any of four attempts passes, but this masks cross-attempt variability. Decomposing into *aptitude* (any attempt passes) and *full consistency* (all four pass), the reliability ratio drops from 67% at round 1 to 20%

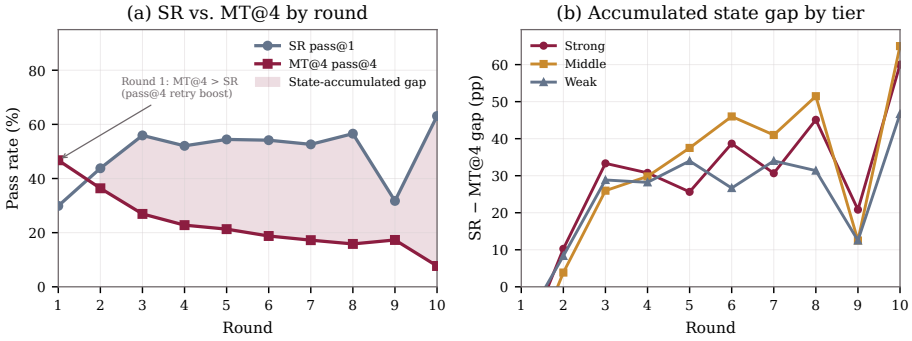

Figure 4: Per-round SR (single attempt) vs. MT@4 (best of four attempts). (a) SR remains stable while MT@4 declines; the shaded region is the state-accumulated gap. (b) SR–MT@4 gap by agent tier; late-round fluctuations reflect small task counts.

at round 5 (Appendix G.9), echoing [4]: multi-turn settings degrade consistency more sharply than capability. The location of first failure also differs by tier: 57.4% of lower-tier trial failures occur in the first 20% of rounds, compared with 29.4% for mid-tier and 14.4% for top-tier agents (Appendix G.12), consistent with the view that lower-tier agents lack basic requirement satisfaction while higher-tier agents survive longer before encountering destabilizing corrections or conflicts.

4.3 Fine-Grained Analysis

Failure patterns are tier-dependent. We label each executed failed round by its primary failure mode: missed active requirement, environment/tooling failure, regression, stale superseded behavior, or context loss. Missed-requirement dominates every tier (87–90%), so the more informative diagnostic is *which secondary modes emerge and when*. Mid-tier failures include stale behavior (28.6%) from superseded requirements left in the codebase; top-tier failures at advanced rounds include regression (18.2%) where previously passing tests break after new edits. Per-round profiles (Appendix G.10) show that for top-tier agents regression peaks at round 2 (35%) and for mid-tier agents conflict-mishandling appears at round 5 (23%), while lower-tier agents are dominated by missed requirements throughout.

These tier differences are partly a selection effect: lower-tier agents fail before encountering the correction and conflict rounds where stale-behavior and regression failures become possible, so their failure distribution is truncated rather than qualitatively different. Mid-tier agents survive enough rounds to encounter specification supersession but fail to update already-implemented behavior accordingly, suggesting that specification tracking across accumulating requirements is a more prominent bottleneck than coding ability at this tier. Top-tier agents survive the longest but show regression when editing code to satisfy new requirements, consistent with modifications that are locally correct but disrupt previously satisfied behavior, a failure surface that only appears when the agent has built enough working functionality to break.

Appendix D.3 defines the taxonomy; Appendix E presents case studies.

Taxonomy-level patterns. Across interaction styles, document-driven tasks average 50.1 MT@4, roughly 2.4× the mean of explorative (20.9) and contractual (20.7) tasks. Across engineering activities, review-driven improvement scores highest (29.6) while specification evolution scores lowest (21.8); leaders also vary by activity, with Opus-4.7 leading construction and review, GPT-5.5 migration, and Opus-4.6 specification evolution. These patterns are diagnostic rather than stable rankings because small cells and task content co-vary with taxonomy labels. When change-type effects are controlled for round position (Appendix G.14), extension rounds show notably higher MT@4 pass rates than correction or conflict rounds in early rounds (43.2% vs. 31.9% vs. 25.0%), with the difference diminishing in later buckets, indicating that requirement revision adds difficulty beyond later round position.

Failing rounds are associated with higher token consumption. To control for the confound that later rounds naturally have longer contexts, we compare output tokens of passing and failing trials at the same round index. Across rounds 1–9 (where both pass and fail samples are sufficient), failing trials produce 1.1–3.1$\times$ as many output tokens as passing trials at the same round (Appendix G.13). The direction of this association is ambiguous: more difficult workspace states may simultaneously cause both failure and increased agent effort, rather than effort itself causing failure. We report this pattern as a diagnostic observation, not a causal claim.

5 Discussion

Section 4 shows that persistent multi-turn coding is not just longer single-round coding: the same instructions are often solvable from a reference-completed state but fail when the agent relies on its own accumulated workspace. This distinction matters as coding agents are deployed as ongoing development partners rather than one-shot patch generators.

Implications for agent system designers. The SR–MT@4 gap implies that reliability cannot be obtained by improving latest-request execution alone. Agents must preserve a running contract over files, tests, dependencies, and earlier corrections. Appendix G.11 shows that 57.0% of multi-turn failures involve rounds solvable from a reference-completed state, rising above 80% at deeper rounds and peaking for correction (73.5%) and conflict (67.1%) rounds. This motivates system support around the model: regression checks between rounds, repository-grounded summaries of active requirements, and workspace audits that detect stale or superseded behavior.

Implications for benchmark designers. EvoCode-Bench also suggests benchmark-design principles that may transfer beyond coding. Cumulative verification is needed because round-local tests miss regressions against still-active requirements. Fail-stop MT@4 and reference-fast-forward SR should be read together: one measures persistent reliability, while the other estimates whether the target round is solvable absent accumulated workspace damage. Behavior-only tests are also important because correct agents may take different implementation paths across attempts.

Implications for model developers. The tier-dependent failures suggest different training bottlenecks. Lower-tier agents need stronger basic requirement satisfaction; mid-tier agents need better specification tracking and conflict resolution; top-tier agents need better regression control during incremental edits. This argues against treating multi-turn coding as only a context-length problem: longer context helps only if the model can identify active requirements, superseded requirements, and earlier implementation choices that have become liabilities. Training on corrections, conflicts, migrations, and cumulative regression checks exercises capabilities that single-turn coding data often hides.

Implications for practitioners and end users. For practitioners, the SR–MT@4 gap suggests that clean workspace restarts or reference-state repair may recover performance, though EvoCode-Bench does not directly evaluate restart strategies. The evaluation consumed approximately 24.4 billion tokens across 3,657 logged multi-round execution records (Appendix G.13), underscoring the cost of persistent multi-turn evaluation. MT@4 and SR should therefore be read together: high SR with much lower MT@4 indicates strength on isolated requests but degradation under sustained iterative development.

6 Conclusion

We introduced EvoCode-Bench, a benchmark for persistent multi-turn coding with cumulative tests, stateful workspaces, and round-level diagnostics. The central finding is that single-round and multi-turn evaluations measure different capability dimensions: the former rewards comprehending and extending an existing codebase, the latter rewards building and maintaining one's own. SR exceeds MT@4 by 22 to 40 points for most agents, with reranking effects in which the highest single-round scorer ranks only third in persistent execution. Only two agents exceed 50 MT@4, and tiered failure patterns separate early misses by lower-tier agents from correction and regression failures among top-tier agents.

References

[1] Anthropic. Claude code. Product documentation, 2026.

[2] Anysphere. Cursor. Product documentation, 2026.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[4] Ahmad Bsharat, Myrzakan Ainur, Jiawei Ye, Hailey Schoelkopf, Jason Phang, Ameya Prabhu, Oam Patel, Ben Allison, Adian Liusie, and Samuel R. Bowman. LLMs get lost in multi-turn conversation. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[6] Le Deng, Zhonghao Jiang, Jialun Cao, Michael Pradel, and Zhongxin Liu. NoCode-bench: A benchmark for evaluating natural language-driven feature addition, 2025.

[7] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2026.

[8] Yaxin Du, Yuzhu Cai, Yifan Zhou, Cheng Wang, Yu Qian, Xianghe Pang, Qian Liu, Yue Hu, and Siheng Chen. Swe-dev: Evaluating and training autonomous feature-driven software development. *CoRR*, abs/2505.16975, 2025.

[9] Mrigank Garg, Vansh Agrawal, Siddarth Ravishankar, Markus Flicke, Angela Zhou, Jingbo Shang, and Frederic Sala. SlopCodeBench: A benchmark for evaluating code quality degradation in iterative LLM-assisted development. *arXiv preprint arXiv:2603.24755*, 2025.

[10] Nam Le Hai, Dung Manh Nguyen, and Nghi D. Q. Bui. On the impacts of contexts on repository-level code generation, 2025.

[11] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments. Open-source evaluation framework, 2026.

[12] Harbor Framework Team. Terminus-2. Harbor reference agent documentation, 2026.

[13] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[15] Wai-Chung Kwan, Xingshan Zeng, Yuxin Jiang, Yufei Wang, Liangyou Li, Lifeng Shang, Xin Jiang, Qun Liu, and Kam-Fai Wong. MT-eval: A multi-turn capabilities evaluation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 20153–20177, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[16] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. In *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 24-28, 2026*. OpenReview.net, 2026.

[17] Jia Li, Ge Li, Xuanming Zhang, Yihong Dong, and Zhi Jin. EvoCodeBench: An evolving code generation benchmark aligned with real-world code repositories, 2024.

[18] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, and Yongbin Li. DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories, 2024.

[19] Tianyang Liu, Canwen Xu, and Julian J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[20] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[21] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, Findings of ACL, pages 1160–1183. Association for Computational Linguistics, 2025.

[22] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.

[23] OpenAI. Codex. Product documentation, 2026.

[24] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.

[25] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): from tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.

[26] Ruchit Rawal, Jeffrey Yang Fan Chiang, Chihao Shen, Jeffery Siyuan Tian, Aastha Mahajan, Tom Goldstein, and Yizheng Chen. Benchmarking correctness and security in multi-turn code generation, 2025.

[27] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 16022–16076. Association for Computational Linguistics, 2024.

[28] Sizhe Wang, Zhengren Wang, Dongsheng Ma, Yongan Yu, Rui Ling, Zhiyu Li, Feiyu Xiong, and Wentao Zhang. CodeFlowBench: A multi-turn, iterative benchmark for complex code generation, 2025.

[29] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[30] Windsurf. Windsurf. Product documentation, 2026.

[31] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *CoRR*, abs/2407.01489, 2024.

[32] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[33] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[34] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. Swe-bench multimodal: Do AI systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[35] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[36] Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Wei Li, Yafen Yao, Yongshun Gong, Xiaolin Chen, Bei Guan, Zhiguang Yang, Yongji Wang, Qianxiang Wang, and Lizhen Cui. CodeS: Natural language to code repository via multi-layer sketch, 2024.

[37] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In Maria Christakis and Michael Pradel, editors, *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, pages 1592–1604. ACM, 2024.

[38] Wentao Zhao, Jingyu Ke, Runchu Tian, Haotian Chen, Liangdong Wang, Jiaheng Liu, Guowei Xu, Zhaoxiang Zhang, Bo Zheng, Wangchunshu Zhou, Ke Liu, and Ge Zhang. SWE-EVO: Benchmarking software-engineering agents on evolving repositories through release-level tasks. *arXiv preprint arXiv:2512.18470*, 2025.

[39] Kunhao Zheng, Juliette Decugis, Jonas Gehring, Taco Cohen, Benjamin Negrevergne, and Gabriel Synnaeve. What makes large language models reason in (multi-turn) code generation? In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-chat-1m: A large-scale real-world LLM conversation dataset, 2023.

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[42] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

A Limitations and Future Work

The benchmark contains 26 tasks; scaling is primarily constrained by the cost of manual instruction authoring, cumulative test construction, cross-validation, and multi-agent evaluation. The fail-stop model may underestimate agents capable of recovery, though the SR metric partially addresses this. More fundamentally, EVOCODE-BENCH covers only tasks for which correctness can be verified through executable tests; evaluating agent reliability on subjective design decisions, code quality judgments, and collaborative negotiations remains an open problem.

A natural next step is adaptive multi-turn evaluation: an examiner agent that maintains a sufficiently complex target system specification and dynamically generates instructions adapted to the evaluated agent's current workspace state and performance trajectory. Such a framework would remove the fixed round structure and let the evaluation adapt its difficulty and direction to expose each agent's failure boundary, enabling more efficient and scalable multi-turn assessment without the bottleneck of fully manual task construction. Extending this approach beyond coding to persistent multi-turn evaluation in other tool-using domains, where each turn modifies a shared, long-lived artifact, is an open challenge.

B Task Format and Reproducibility Details

Section 3 defines the benchmark semantics: multi-round task execution, cumulative verification, fail-stop scoring, and the two-dimensional taxonomy. This appendix provides the exact release format and execution details needed for reproducibility. All tasks follow the same directory layout, metadata schema, and execution protocol described below.

B.1 Released Task Layout

Each task is a self-contained directory with one environment definition and N round subdirectories:

```
task/
  task.toml                    # Metadata
  environment/
    Dockerfile                 # Shared runtime environment
  round_1/
    instruction.md             # Round 1 instructions
    solution/solve.sh          # Round 1 reference solution
    tests/test.sh              # Round 1 verification (cumulative)
  round_2/
    instruction.md
    solution/solve.sh
    tests/test.sh
  ...
  round_N/
    instruction.md
    solution/solve.sh
    tests/test.sh
```

The `Dockerfile` in the `environment/` directory defines a reproducible runtime that includes all language runtimes, system libraries, and tool dependencies needed by the task. Each round's `instruction.md` is the verbatim instruction delivered to the agent. The `solve.sh` script applies the reference solution for that round only, and the `test.sh` script runs the cumulative verifier.

B.2 Task Metadata Format

The `task.toml` file records all metadata needed to configure evaluation:

```
[metadata]
name = "protocol-parser-evolution"
num_rounds = 5
```

```
engineering_activity = "spec_evolution"
interaction_style = "document_driven"
change_types = [
  ["extension"],
  ["extension", "correction"],
  ["conflict"],
  ["conflict", "extension"],
  ["extension"]
]
technology_domain = "system_protocol"
has_agents_md = false
```

The `change_types` array records the requirement-change labels for each round. Valid labels are **extension** (adding new capabilities without altering existing behavior), **correction** (adjusting previously established behavior without overturning core assumptions), and **conflict** (overturning a core assumption and requiring structural adaptation). A single round may carry multiple labels when the instruction combines, for example, an extension with a correction. These labels are used for the diagnostic analyses in Section 4.3 and for the change-type breakdowns in Appendix G.

B.3 Execution Semantics

All rounds execute within a single Docker container. The evaluation framework builds the Docker image from the task's `Dockerfile`, initializes one agent session for the entire task, and then executes each round sequentially: delivering the instruction, waiting for the agent to signal completion, running the cumulative test script, and recording the binary reward. If any round fails ($r_i = 0$), the task terminates immediately under the fail-stop protocol described in Section 3.1. After all rounds complete, the composite score is computed as $S = \frac{1}{N}\sum_{i=1}^{N} r_i$. This execution model ensures that every evaluated round depends on the workspace state produced by the same agent in all preceding rounds, which is the defining property of persistent multi-turn evaluation.

B.4 Reference Solution Semantics

Each round's `solve.sh` makes only that round's incremental changes. Executing `solve.sh` for rounds 1 through N in sequence produces the complete reference state after round N. Reference solutions must never rewrite the entire project; they build incrementally on the prior state, mirroring the incremental development process that the benchmark requires of evaluated agents. This constraint is verified during oracle verification (Section 3.2): each reference delta is applied in sequence, and the cumulative verifier must pass at every round boundary.

B.5 Cumulative Test Semantics

Round i's `test.sh` verifies all still-valid requirements from rounds 1 through i. When a later round's conflict or correction supersedes an earlier requirement, the cumulative test is updated accordingly: assertions that check the now-obsolete behavior are replaced by assertions that check the new behavior. Tests run all assertions to completion without early termination on the first failure, producing a comprehensive pass/fail report that supports diagnostic failure annotation (Appendix D.3). Test scripts verify behavior through actual system execution only: they invoke the built system, send inputs, check outputs, and compare results. They never inspect source code structure, function names, file organization, or class hierarchies, ensuring that the verifier accepts any behaviorally correct implementation regardless of internal design choices.

C Task Examples

Section 3.3 reports that EVOCODE-BENCH contains 26 tasks spanning 227 evaluated rounds. This appendix presents one released task in abbreviated form to illustrate the cumulative verification design and the cross-round dependencies that distinguish multi-turn evaluation from concatenated single-turn tasks.

C.1 Released Task: Deterministic Data Pipeline CLI

The task `deterministic-data-pipeline-go` is a 15-round contractual construction task in the data-engineering-reproducibility domain. The agent must build and extend `dpipe`, a deterministic Go CLI for CSV ingestion, transformation, validation, profiling, lineage tracking, quality checks, snapshots, and changelogs. It is representative because it combines cumulative feature growth, behavior corrections, and explicit conflicts with earlier output formats. Table 3 shows an abbreviated round chain.

Table 3: Abbreviated round chain from the released task `deterministic-data-pipeline-go`. The omitted rounds add commands such as profiling, validation, audit logging, schema inspection, fingerprinting, snapshots, and changelogs.

Round	Change type	Instruction summary
1	Extension	Build `dpipe` as a deterministic Go CLI that reads CSV files, validates rows against JSON schemas, applies ordered transformations, writes bit-identical outputs, and records reproducibility metadata.
3	Extension, correction	Correct linear-fill boundary behavior and z-score normalization while adding extended profiling and a new `drift` command. The verifier checks both the new behavior and the earlier deterministic pipeline.
5	Extension, conflict	Change the `verify` and `manifest` checksum behavior from SHA-256 sidecar files to BLAKE2b-256 manifest checksums, while adding a `lineage` command.
7	Extension, correction	Correct lineage semantics for manifest steps: directory scans should not become input nodes. Add a temporal transform for time-series operations.
10	Extension, conflict	Replace the manifest entry field `blake2b` with `checksum` and `algorithm`, then add a `reconcile` command that compares two manifests.
12	Extension, correction	Correct the default `normalize` method so an omitted or empty method behaves as `minmax`. Add a numeric `round` transform.
15	Extension, conflict	Change `snapshot` row hashes from MD5 to SHA-256 and add a constant-value `tag` transform. The final verifier exercises the accumulated system across transforms, auditability, lineage, manifest formats, snapshots, and changelogs.

This example illustrates why the benchmark uses cumulative verification. A correct round 15 submission must do more than implement the latest hash-format change. It must also preserve deterministic output, earlier checksum semantics where still active, profile/validate/quality/schema behavior, audit logging, lineage metadata, and snapshot/changelog requirements introduced across the previous rounds. This is the evaluation setting targeted by EvoCode-Bench: later instructions may look local, but the verifier scores the whole evolving system.

C.2 Document-Driven Constraints

Section 3.1 introduces document-driven interaction as one of the three interaction styles. In document-driven tasks, persistent requirements are encoded in project artifacts such as `AGENTS.md` or specification files rather than in the chat-based instruction stream. The instruction for a given round may simply state that the specification file has been updated, without repeating the changed requirements in the instruction text. The cumulative verifier then checks whether the agent's implementation is synchronized to the updated artifact.

This design tests a capability that purely chat-centric evaluation cannot measure: whether the agent reads, interprets, and preserves repository-level constraints across rounds. In practice, the benchmark's four document-driven tasks each contain an evolving specification document. A round may add a new constraint, tighten an existing tolerance, or remove a previously required behavior through a specification update. The verifier checks the implementation against the current state of the specification, not against the instruction text alone. Agents that rely exclusively on the latest chat message and ignore changes to project files are expected to fail on these tasks, particularly after specification updates that are not echoed in the instruction.

D Quality Assurance Details

Section 3.2 describes the four-stage quality pipeline used to construct EVOCODE-BENCH tasks. This appendix provides additional detail on the design principles, cross-validation protocol, and failure annotation methodology.

D.1 Design Principles

Three principles govern all task construction in EVOCODE-BENCH. These principles address the core challenge of multi-turn benchmark design: different agents may reach equivalent behavioral goals through different implementation paths, so neither instructions nor tests can assume a specific internal design.

1. **Describe behavior, not implementation.** Instructions specify what the system should do in terms of observable behavior, not how to achieve it. No algorithm names, data structure choices, or file organization prescriptions appear in instructions. This lets different agents reach equivalent behavioral goals through different implementation paths.
2. **Test behavioral requirements, not implementation details.** Verification scripts check the system's external behavior through actual execution: running commands, sending inputs, and checking outputs. They never inspect source code structure, function names, or file organization. This makes the same test suite valid across arbitrary implementation paths.
3. **Round-independent testing.** Round i's tests cannot assume the agent used the same code structure as the reference solution in rounds 1 through $i-1$. They can only assume the agent passed the behavioral tests of all prior rounds. This is the key mechanism for handling divergent implementation paths in multi-turn evaluation.

These principles are enforced at every stage of the pipeline: during task authoring, during internal review, during oracle verification, and during final cross-validation. Violations discovered at any stage trigger task revision before the task enters the benchmark.

D.2 Cross-Validation Protocol

After tasks pass internal review and oracle verification, a final cross-validation stage addresses three objectives that go beyond checking whether the reference solution is correct:

1. **Answer correctness verification.** Independent reviewers verify that the reference solution for each round produces the behavior specified by the instructions. They check for alternative valid interpretations that the tests might not distinguish and for edge cases that the cumulative tests might miss.
2. **Test-specification alignment audit.** Reviewers verify that every test assertion traces back to a current specification, either newly introduced in the current round or persisting from a previous round. They also verify that no test checks stale requirements that have been superseded by later rounds, which would cause otherwise correct implementations to fail.
3. **Shortcut analysis.** Reviewers attempt to identify implementation strategies that would pass all tests without satisfying the task requirements. This includes checking for overly specific test inputs that allow hardcoding, test orderings that leak information, and behavioral checks that are too loose to distinguish correct from incorrect implementations. Tasks with identified shortcuts are revised to close the gap.

Tasks that cannot be brought into alignment across all three objectives are removed from the benchmark. This conservative filtering contributes to the benchmark's relatively small size (26 tasks) but ensures that each released task provides a controlled evaluation trajectory.

D.3 Failure Annotation Protocol

Section 4.3 reports tier-dependent failure patterns. To produce these diagnostics, we build one evidence packet for each executed multi-round fragment whose verifier reward is zero. We do not classify implicit zeros from unexecuted future rounds (which result from fail-stop termination). Each packet contains the model identity, task taxonomy labels, failed round index, change types, current and previous instructions, verifier stdout/stderr, test exit code, cumulative test script excerpts, reference-solution excerpts, and paths back to the raw Harbor result. Verifier output is placed first

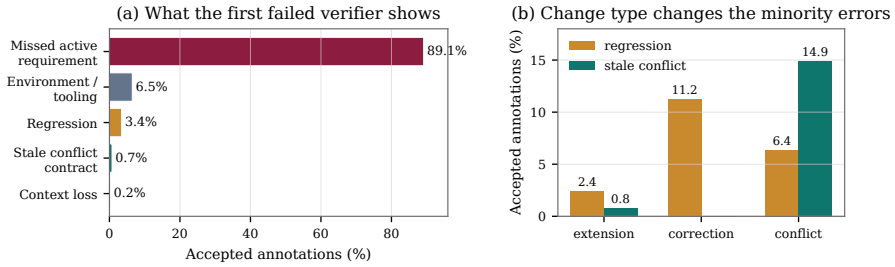

Figure 5: Failure diagnostics expanded from Section 4.3. (a) The primary accepted label for each executed failed round fragment. (b) Regression and stale-conflict labels by task-metadata change type, showing why correction and conflict rounds expose different failure surfaces.

in the annotation prompt so that concrete assertion failures remain visible under context-length truncation.

An LLM annotator assigns one primary label, optional secondary labels, a confidence score, and short supporting evidence. The five reported labels are:

- *Missed active requirement*: the failed assertions correspond to behavior newly required or still required at the failed round, and the submitted workspace does not implement that behavior.
- *Environment/tooling failure*: package installation, build setup, command wiring, generated artifacts, permissions, or runtime invocation prevents the verifier from exercising the intended behavior.
- *Regression*: behavior that had become active in an earlier round is broken by a later edit, even though the latest instruction may have been partially implemented.
- *Stale superseded behavior*: a conflict round explicitly replaces earlier behavior, but the workspace continues to expose the obsolete behavior.
- *Context loss*: the evidence shows that the agent ignored necessary prior task state, files, or corrected semantics; this label is used only when the failure is not better explained as a local missed requirement.

The prompt requires `unknown` when logs do not identify a root cause. Low-confidence, malformed, timeout, and `unknown` annotations are retained in a review queue and excluded from aggregate failure-mode percentages. The accepted labels are therefore used as evidence-backed qualitative diagnostics, not as an additional scoring metric.

Human validation. All failure annotations from the multi-turn evaluation were manually reviewed by the authors; every accepted label was confirmed against the verifier output and agent trajectory. For the single-round evaluation, we sampled 50 annotated failures uniformly across tiers and failure labels and independently re-annotated them. The human labels agreed with the LLM annotations on all 50 cases (100% agreement), indicating that the structured evidence packets provide sufficient information for reliable classification.

Figure 5 expands the tier-level failure diagnostics from Section 4.3 into per-label and per-change-type breakdowns.

E Detailed Case Studies

Section 4.3 reports that failure patterns differ by agent tier and that the gap between SR and MT@4 is associated with persistent-state dependency. This section presents three case studies using released task artifacts to illustrate these observations concretely. The examples address three audit questions: what makes a late round depend on earlier rounds, why SR differs from MT on the same instruction, and what information fail-stop scoring preserves.

E.1 Case Study 1: Late-Round Conflict in a Released 15-Round Task

Task and audit question. The released task `deterministic-data-pipeline-go` asks the agent to build `dpipe`, a deterministic Go CLI for data-pipeline reproducibility. Table 3 gives an abbreviated

round chain. The main audit question is whether the final rounds are independent feature additions. They are not: later rounds revise checksum formats, lineage semantics, normalization defaults, snapshot hashes, and transforms while still requiring the earlier deterministic pipeline, validation, profiling, audit, schema, fingerprint, snapshot, and changelog behavior. Table 4 traces the active requirements through selected rounds.

Table 4: Active requirements in `deterministic-data-pipeline-go`. The final round is local in wording but global in verification: the cumulative verifier tests all active requirements, not only the latest addition.

Round	Active requirement added or revised	Why it still matters later
1	Deterministic CSV pipeline and schema validation	All later commands operate on the same deterministic data model and must preserve bit-identical output behavior.
3	Boundary fill and z-score corrections; profiling and drift	Later transforms are checked against corrected numerical semantics, not only against newly added commands.
5	Replace SHA-256 sidecars with BLAKE2b-256 manifest checksums; add lineage	Later manifest, reconcile, and audit behavior depends on this checksum migration and on lineage records.
7	Correct lineage graph semantics; add temporal transform	Directory scans must not become false input nodes, and later lineage checks inherit this rule.
10	Replace manifest field `blake2b` with `checksum` and `algorithm`; add reconcile	A correct later state must stop emitting the obsolete field while preserving manifest comparability.
12	Correct default `normalize` behavior; add numeric rounding	Later transform tests use the corrected default and identical-value behavior as part of the accumulated requirements.
15	Change snapshot row hashes from MD5 to SHA-256; add constant-value tags	The final verifier checks the new snapshot hash behavior together with the accumulated transforms, lineage, manifest, audit, and changelog behavior.

Why the final round is not a local edit. Round 15 asks for a snapshot hash change and a tag transform. A solution that only edits the latest snapshot routine can still fail if it leaves the round-10 manifest schema inconsistent, restores obsolete SHA-256 sidecars from round 5, forgets the round-7 lineage correction, or regresses the round-12 normalization rule. The cumulative verifier operationalizes the claim behind MT@4: an agent must maintain an evolving workspace whose active requirements are distributed across earlier turns. This cross-round dependency is what makes persistent multi-turn evaluation distinct from a sequence of independent coding prompts.

E.2 Case Study 2: Why SR Does Not Imply MT Reliability

Controlled comparison. SR and MT@4 evaluate the same round instructions under different workspace conditions. In SR, Harbor fast-forwards the workspace with reference deltas before handing the target round to the evaluated model. In MT@4, the model receives the next instruction in the workspace it produced itself. The difference is visible on tasks such as `deterministic-data-pipeline-go`: a round-10 single-round attempt starts from a reference implementation that already satisfies the BLAKE2b migration and lineage correction, while a multi-round attempt must carry those prerequisites forward from its own earlier edits. This controlled comparison isolates the effect of accumulated workspace state on task success.

Concrete failure surface. The round-10 request replaces the manifest field `blake2b` with the pair `checksum` and `algorithm`. In SR, the model can focus on the schema rename and the new reconcile command, because the reference workspace already contains a correct BLAKE2b manifest implementation. In MT@4, the same edit is entangled with whether the model's prior implementation already removed SHA-256 sidecars, whether manifest generation is deterministic, and whether lineage entries remain consistent. SR therefore measures isolated instruction-following skill; MT@4 measures whether earlier work remains a usable foundation for later work.

Metric implication. This distinction explains why Table 2 reports both MT@4 and SR. A high SR with a substantially lower MT@4 is not contradictory: it means that the agent can often solve

individual rounds from a reference state while losing reliability when later rounds depend on its own accumulated workspace. Agents with a small SR–MT@4 gap and high SR are those whose persistent workspaces closely approximate reference-quality states. This gap is the empirical basis for the claim in Section 4.2 that persistent execution amplifies the difference across agent tiers.

E.3 Case Study 3: What Fail-Stop Localizes

Audit question. Fail-stop scoring is useful only if the failed round provides interpretable evidence. In EVOCODE-BENCH, the failed round is the first round where the agent-produced workspace no longer satisfies the cumulative verifier. Annotators then inspect which assertions failed and whether they correspond to newly introduced behavior (missed active requirement), still-active prior behavior (regression), or superseded behavior that should have been removed (stale conflict). Table 5 illustrates each label with concrete examples from the data-pipeline task.

Table 5: How fail-stop traces are interpreted during failure annotation. Each label corresponds to a distinct failure mechanism visible in the verifier output.

Failure label	Example evidence in the data-pipeline task
Regression introduction	A later manifest or snapshot edit breaks deterministic output, schema validation, profiling, or audit behavior that had already been required.
Conflict mishandling	The workspace keeps an obsolete field such as `blake2b`, keeps writing removed checksum sidecars, or continues using MD5 row hashes after SHA-256 becomes the active requirement.
Context loss	Corrected details such as default `normalize` behavior or lineage graph rules disappear from later edits.
Environment mismanagement	The Go CLI no longer builds, generated artifacts become stale, or command wiring changes in a way that prevents cumulative tests from exercising the tool.

Why the localization matters. Continuing after a failure would produce additional observations, but those observations would run on a workspace that is already known to violate active requirements. The resulting data conflates two questions: whether the agent can solve the next instruction, and whether it can repair accumulated damage. Fail-stop instead records the first requirement failure and leaves later-round ability to SR or reference-fast-forward analyses. This separation is why the paper reports MT@4 for persistent reliability, SR for isolated round skill, Comp for full-task completion, and round-k pass rates for where failures appear along the horizon. The diagnostic value of each metric depends on the others: MT@4 alone does not reveal whether failures cluster early or late, while round-k pass rates alone do not reveal whether failures reflect persistent-state issues or instruction difficulty.

F Evaluation Infrastructure Details

Section 3.4 describes the evaluation protocol at a design level. This appendix provides the infrastructure details: the agent harness, model configuration, Harbor extensions, reproducibility protocol, and alternative scoring designs that were considered but not adopted.

F.1 Terminus-2 Agent Harness

Terminus-2 [12] is a lightweight agent harness that connects language models to the evaluation container through a uniform interface. It provides each model with shell access to the Docker container, supporting file reads and writes, command execution, package installation, and test execution. At each round, Terminus-2 delivers the round's instruction to the model, relays the model's tool calls to the container, and collects the model's completion signal. The harness imposes no constraints on the model's task-solving strategy: it may read code, run tests, install dependencies, or execute arbitrary shell commands. All models receive the same system prompt and tool definitions, reducing harness configuration as a source of variation.

Interaction and token accounting. **Avg. Turns** estimates main-agent Terminus-2 episodes per full multi-round agent-task evaluation. For each recorded trajectory, the summarizer sums `agent_result.metadata.n_episodes` over fragments, divides by the number of covered rounds, and multiplies by the task's total number of rounds; complete executions use the observed episode count directly. If `n_episodes` is absent, the summarizer falls back to the number of recorded request-time entries. **Output Tok. (K)** is computed by summing `agent_result.n_output_tokens` over recorded fragments, averaging trajectory totals, and dividing by 1000. Terminus-2 obtains token usage from LiteLLM response metadata. Chat-completion calls provide prompt and completion token counts; Responses API calls provide input and output token counts. Harbor accumulates input tokens, output tokens, cache-hit input tokens, and cost over the agent run. The paper reports output tokens only. This value is provider-reported generated-token usage rather than a tokenizer pass over saved transcripts; hidden reasoning tokens are included only when the provider includes them in the reported completion tokens. Harbor does not separately add provider-specific reasoning-token detail fields to `n_output_tokens`.

F.2 Model Configuration

All 13 models are accessed through API-compatible endpoints configured in the evaluation model configuration file. No model receives a custom task prompt beyond the standard Terminus-2 harness prompt, and no model-specific prompt engineering is applied. The main text, tables, and figures use compact labels to fit dense results; Table 6 gives the full evaluated names and endpoint identifiers.

Table 6: Mapping from compact labels used in the paper to the full evaluated model configurations. All models use the same Terminus-2 harness and Harbor evaluation protocol.

Paper label	Full evaluated name	Endpoint model string	Evaluation setting
Opus-4.7	Claude-Opus-4.7-High	`openai/claude-opus-4-7`	high reasoning effort
GPT-5.5	GPT-5.5-High	`openai/gpt-5.5`	high reasoning effort
Opus-4.6	Claude-Opus-4.6	`openai/claude-opus-4-6`	default configured reasoning
GLM-5.1	GLM-5.1	`openai/z-ai/glm-5.1`	thinking enabled
Kimi-K2.6	Kimi-K2.6	`openai/moonshotai/kimi-k2.6`	thinking enabled
DS-V4-Pro	DeepSeek-V4-Pro	`openai/deepseek/deepseek-v4-pro`	high reasoning effort
Qwen3.6-Plus	Qwen3.6-Plus	`openai/qwen/qwen3.6-plus`	thinking enabled
MiMo-V2.5	Xiaomi-MiMo-V2.5-Pro	`openrouter/xiaomi/mimo-v2.5-pro`	high reasoning effort
Gemini-3.1	Gemini-3.1-Pro-Preview	`openai/gemini-3.1-pro-preview`	high reasoning effort
DS-V4-Flash	DeepSeek-V4-Flash	`openai/deepseek/deepseek-v4-flash`	high reasoning effort
Qwen3.5-397B	Qwen3.5-397B-A17B	`openai/qwen3.5-397b-a17b`	thinking enabled
MiniMax-M2.7	MiniMax-M2.7	`openai/minimax/minimax-m2.7`	reasoning split enabled
Doubao-2.0	Doubao-Seed-2.0-Pro	`openai/doubao-seed-2.0-pro`	high reasoning effort

F.3 Harbor Multi-Turn Extensions

The Harbor framework [11] was originally designed around a single instruction, a single agent execution, and a single verification step. EVOCODE-BENCH requires different execution semantics: the same workspace and agent session must persist across rounds, while the verifier changes at each round and continues to check all still-valid requirements. We therefore extend Harbor at the level of task semantics and trial state. The extensions are summarized below; each addresses one aspect of the persistent multi-turn protocol.

Round as the unit of instruction and verification. A multi-round task declares `num_rounds` and one metadata record per round. Each round has its own instruction, reference delta, test script, and change-type labels. Harbor validates that the declared rounds are contiguous and that every round contains the required files. This keeps the task format explicit: there is no hidden convention that infers rounds from file names alone.

Persistent execution with explicit round boundaries. The agent works in one Docker container and one agent session across the whole task. At each boundary, Harbor delivers the next instruction, waits for the agent to finish, replaces the verifier payload with that round's cumulative tests, records the binary reward, and then either advances or stops. The agent sees a natural sequence of user requests; the evaluation system owns the round boundary, verifier swap, reward recording, and fail-stop decision.

Cumulative tests and fail-stop aggregation. Harbor runs the cumulative verifier for round i from `round_i/tests/`, checking all requirements from rounds 1 through i that remain active. The trial result stores both per-round rewards and the aggregate window used for scoring. Missing rewards inside the scoring window count as zero, which gives the fail-stop score $S = \frac{1}{N}\sum_i r_i$ while preserving the detailed round log for analysis.

Reference fast-forward for controlled comparisons. Some analyses require testing a target round from a known-correct state. Harbor supports this by applying the reference deltas for rounds before the target round, then handing the workspace to the evaluated agent. This protocol is used for single-round evaluation (SR). It is not used for the main multi-turn score, because the main score must depend on the agent's own earlier work.

Resume and state lineage. Long multi-round evaluations need recovery from infrastructure interruption without changing the evaluation semantics. Harbor therefore records round-boundary state snapshots, agent-session snapshots, per-round verifier outputs, and lineage metadata linking a resumed trial to its source trial and completed round. Resume is constrained by agent identity and task checksum checks, so a resumed run is a continuation of a documented prior state rather than a new evaluation condition.

Separation of execution and scoring windows. The execution window determines which rounds are actually run. The scoring window determines which round rewards contribute to the aggregate score. Keeping these windows separate supports late-round debugging, round-isolated evaluation, and ablation studies while leaving the main benchmark definition fixed.

F.4 Reproducibility Protocol

Each evaluation run produces a structured output directory that records all information needed to reproduce and audit the result:

```
trial/
  config.json         # Complete run configuration
  round_1/
    reward.json       # Binary reward and verification log
    state/checkpoint/ # Optional container checkpoint
    trajectory.jsonl  # Agent actions and observations
  round_2/
    ...
  round_N/
    ...
  aggregate.json      # Composite score and scoring window
```

The `config.json` records the model identifier, agent harness version, Docker image hash, random seeds, and all evaluation parameters. The `aggregate.json` records the scoring window boundaries and the composite score formula, enabling exact reproduction of any reported result. The `trajectory.jsonl` files contain the full sequence of agent actions, tool calls, and observations, supporting post-hoc analysis of agent behavior at any round.

F.5 Scoring Alternatives

Section 3.1 describes fail-stop scoring as the primary mechanism and explains why it is appropriate for evaluating persistent multi-turn reliability. Two alternative designs were considered during benchmark development; we describe them here along with the reasoning behind their rejection.

Continue-after-failure. Under this design, the agent continues to receive instructions after a failed round. This approach would yield more data points per task but introduces a confound: later rounds execute on a workspace that already violates active requirements, so their results reflect the interaction between the current instruction and the accumulated damage from prior failures rather than the agent's ability on that instruction in isolation. In multi-turn coding, a broken build, a corrupted schema, or a missing dependency can cascade through later rounds in ways that obscure the failure's root

cause. We considered this approach unsuitable for diagnostic evaluation, though it may be useful for studying recovery capabilities in future work.

Reference fast-forward. When the agent fails round k, the evaluation framework applies the reference solution for round k, restoring the workspace to a known-good state before proceeding to round $k + 1$. This approach isolates each round's difficulty but removes the defining property of multi-turn evaluation: the dependence of later rounds on the agent's own prior work. It also inflates scores by giving the agent a foundation it did not build. We use reference fast-forward only for the SR metric, not for the primary MT@4 score.

The fail-stop model is a conservative choice that may underestimate agents capable of recovering from errors. However, it provides a clear diagnostic signal: the first failed round is the first point where the accumulated workspace no longer satisfies the cumulative verifier. The round-k pass rate curves reported in Section 4.2 complement the aggregate MT@4 score by revealing the shape of performance degradation, and the SR metric provides a complementary view of isolated round difficulty.

G Supplementary Results and Diagnostics

Section 4.2 reports aggregate results across all 13 agents, and Section 4.3 presents fine-grained failure diagnostics. This appendix provides supplementary breakdowns that support and extend those findings. The released evaluation package includes the full per-task, per-round, and per-category data in machine-readable JSON and CSV formats.

G.1 Per-Task Scores

Table 2 in the main text reports aggregate MT@4, SR, and Comp across all 26 tasks. The released evaluation package provides the corresponding per-task breakdowns: for each agent-task pair, the package includes the per-attempt round-level rewards, the MT@4 score, the single-round pass rates, agent-interaction turns, and output-token usage. These per-task scores reveal substantial variance across tasks even for the strongest agents. For example, Opus-4.7 achieves MT@4 above 80 on several short construction tasks while scoring below 30 on long migration tasks, consistent with the finding in Section 4.3 that task characteristics modulate difficulty beyond aggregate averages.

G.2 Per-Round Pass Rates

Section 4.2 reports that round-level MT@4 pass rates decline sharply within the first few rounds. Table 7 provides the full round-by-round breakdown. Pass rates drop from 46.7% at round 1 (all 26 tasks active) to 36.4% at round 2, 26.9% at round 3, 22.8% at round 4, and 21.3% at round 5. The decline continues more gradually through rounds 6–9 (18.8%, 17.2%, 15.8%, 17.3%). Rounds 10–15 involve progressively fewer tasks (5 tasks reach round 10, 2 tasks reach round 15), so pass rates at these horizons have higher variance and should be interpreted with caution. The fluctuations at rounds 14–15 (11.5%) reflect the small number of tasks rather than a genuine recovery.

G.3 Cost and Horizon Diagnostics

Section 4.2 notes that performance degrades across rounds and that task length is associated with lower MT@4 scores. Figure 6 provides two diagnostic plots. Panel (a) shows that average interaction turns (a proxy for computational cost) are not monotonically associated with MT@4: some low-scoring agents produce many interaction turns without improving outcomes, while some high-scoring agents are relatively efficient. Panel (b) shows that mean MT@4 decreases across task-length buckets, from shorter tasks to longer ones, consistent with accumulated workspace inconsistency contributing to the decline, though task content also co-varies with length. However, because shorter and longer tasks differ in content as well as length, this association should be interpreted as diagnostic rather than causal.

Table 7: Round-level MT@4 pass rates averaged across all 13 agents. The number of active tasks decreases at longer horizons because tasks have different lengths (5–15 rounds).

Round	Mean pass rate (%)	Active tasks
1	46.7	26
2	36.4	26
3	26.9	26
4	22.8	26
5	21.3	26
6	18.8	25
7	17.2	25
8	15.8	17
9	17.3	12
10	7.7	5
11	7.7	5
12	9.6	4
13	9.6	2
14	11.5	2
15	11.5	2

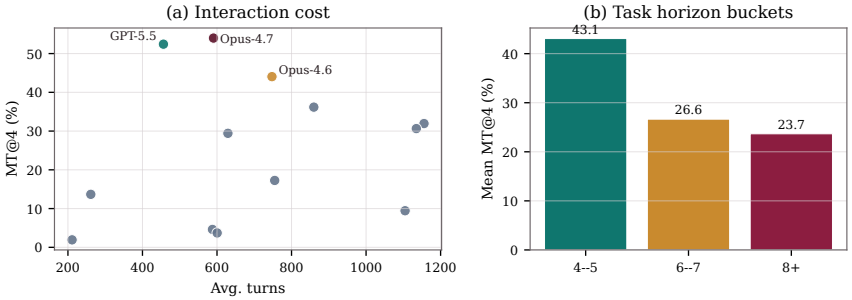

Figure 6: Auxiliary diagnostics for multi-round evaluation. (a) Average interaction turns are not monotonic with MT@4, indicating that more interaction turns do not by themselves explain persistent reliability. (b) Mean MT@4 decreases across longer task-horizon buckets.

G.4 Scores by Technology Domain

EvoCode-Bench spans six broad technology domains: ML and MLOps (9 tasks), data engineering (4 tasks), systems and code (3 tasks), scientific computing (3 tasks), testing and automation (4 tasks), and infrastructure and security (3 tasks). Domain labels are recorded in `task.toml` and are visualized in Figure 2. We intentionally treat domain labels as secondary to the two-dimensional taxonomy (Section 3.1), because technical domain and multi-turn behavior are not the same property: an MLOps task can be construction-oriented, review-driven, or migration-oriented depending on the requirement chain. Domain coverage instead supports external validity by ensuring that the benchmark exercises a range of programming languages, frameworks, and system types rather than concentrating on a single technology stack.

G.5 Scores by Change Type Composition

Section 3.3 reports that 110 distinct rounds carry at least one correction or conflict annotation. The failure annotation data (Appendix D.3) reveals how failure patterns vary across change types. On extension rounds, the dominant failure mode is missed active requirement (90.1% of annotated failures), with environment/tooling failures accounting for 6.5% and regression for 2.4%. On correction rounds, missed active requirement remains dominant (84.8%) but regression rises to 11.2%, consistent with the difficulty of updating previously working behavior. On conflict rounds, the failure distribution is most diverse: missed active requirement drops to 72.3%, conflict mishandling (stale superseded behavior) rises to 14.9%, and both environment/tooling and regression contribute 6.4%

each. This pattern supports the interpretation in Section 4.3 that conflict rounds expose a qualitatively different failure surface from extension rounds.

G.6 Scores by Initial Workspace State

Tasks in EVOCODE-BENCH begin from three workspace conditions: empty or lightly scaffolded projects (construction and specification-evolution tasks), documentation-driven projects with persistent specification files (document-driven tasks), and existing legacy codebases requiring adaptation (migration tasks). The initial workspace state is part of the task design record in `task.toml`. Migration tasks are generally associated with lower MT@4 scores, consistent with the finding that adapting an existing codebase while preserving backward compatibility is more challenging than building from scratch. However, because initial workspace state co-varies with engineering activity and task content, we treat this observation as a task-design characteristic rather than an independent experimental variable.

G.7 Impact of AGENTS.md

Section 4.3 reports that document-driven tasks average 50.1 MT@4 across agents, roughly 2.4× the mean of explorative and contractual tasks. Four of the 26 tasks use `AGENTS.md` or equivalent specification files to encode persistent quality constraints. In these tasks, the instruction may only state that the specification has been updated, and the verifier checks whether the implementation is synchronized to the current document state. The elevated MT@4 on document-driven tasks may reflect the availability of a persistent, readable reference rather than an inherently easier task structure; however, the sample size (4 tasks, 31 rounds) is too small to draw causal conclusions. Expanding the benchmark with additional document-driven tasks across different engineering activities would help determine whether the document-driven advantage generalizes.

G.8 Per-Round SR vs. MT@4 Comparison

Figure 4 (main text) compares single-round pass rates (SR, single attempt from a reference-fast-forwarded workspace) against multi-round MT@4 (best of four persistent-execution attempts) at each round index. SR rates remain stable between 52% and 57% for rounds 3–8, confirming that isolated round difficulty does not increase substantially with round index. In contrast, MT@4 rates decline monotonically from 46.7% at round 1 to 7.7% at round 10. At round 1, MT@4 exceeds SR because MT@4 selects the best of four independent attempts (pass@4 effect) while SR uses a single attempt; this ordering reverses from round 2 onward as persistent-state degradation outweighs the retry advantage. Appendix G.3 provides supporting task-length diagnostics.

G.9 Cross-Attempt Variance and Reliability

Figure 7 decomposes multi-attempt performance into aptitude (at least one of four attempts passes) and full consistency (all four attempts pass). The reliability ratio (full consistency divided by aptitude) drops from 67% at round 1 to 20% at round 5, indicating that cross-attempt consistency declines at deeper rounds. At round 1, 31.4% of (agent, task) pairs pass all four attempts; by round 3, only 7.7% do, even though 26.9% still pass at least once. This decomposition parallels the aptitude-vs-unreliability framework of [4] and shows that the reliability gap appears across all evaluated models: per-model breakdowns reveal that even the strongest agents exhibit declining reliability ratios at deeper rounds.

G.10 Failure Mode Progression by Round

Figure 8 shows how the distribution of annotated failure modes changes across rounds for each agent tier. Top-tier agents exhibit a regression peak at round 2 (35% of 17 failures), coinciding with the first correction rounds that interact with initial implementations; at later rounds, missed-requirement failures dominate as the number of top-tier failures decreases. Mid-tier agents show conflict-mishandling failures emerging at round 5 (23% of 22 failures), absent from rounds 1–4, coinciding with the first rounds where superseded requirements accumulate. Lower-tier agents fail predominantly through missed requirements at every round (>85%), with regression appearing only

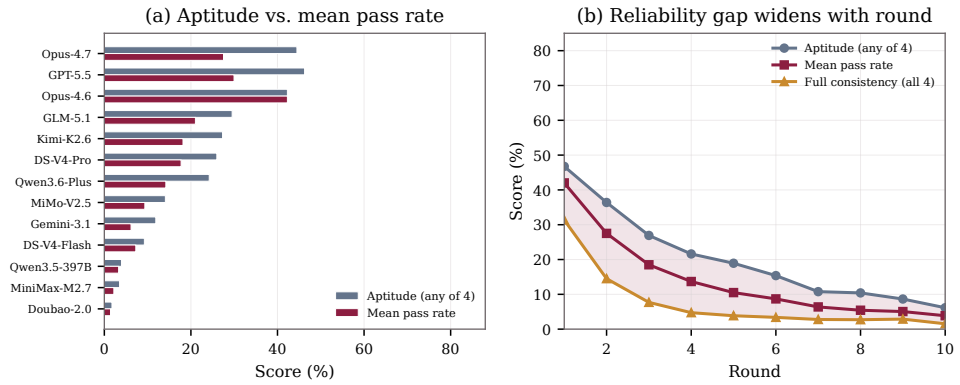

Figure 7: Cross-attempt variance decomposition. (a) Per-model aptitude (any attempt passes) versus full consistency (all attempts pass) and mean pass fraction. (b) Per-round decomposition showing aptitude, mean pass rate, and full consistency; the shaded region between aptitude and full consistency is the reliability gap.

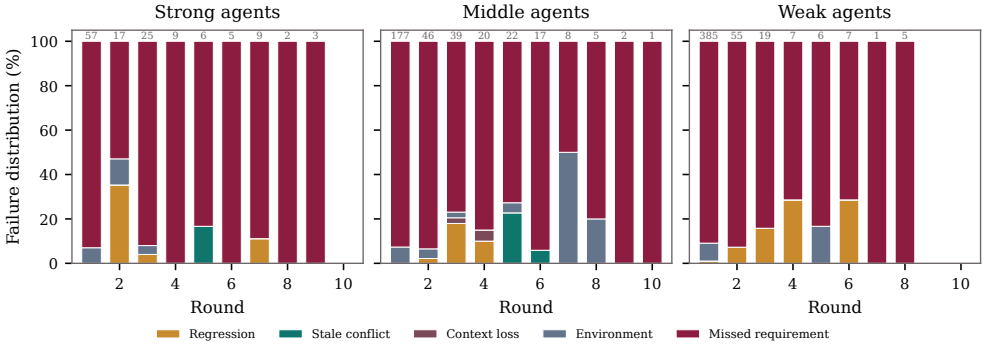

Figure 8: Failure mode distribution by round index for each agent tier. Bar heights show the percentage breakdown of annotated failure modes; numbers above bars indicate the failure count at that round. Regression and conflict-mishandling failures are more visible at intermediate rounds for top-tier and mid-tier agents, while lower-tier agents are dominated by missed-requirement failures throughout.

sporadically (7–29% at rounds 2–6 on very small sample sizes). The count annotations atop each bar show that most failures concentrate at rounds 1–3 across all tiers.

G.11 Per-Round Workspace State Penalty

Section 4.2 reports that SR remains stable while MT@4 declines across rounds, with the gap reaching 41 points by round 8. This appendix quantifies the workspace state penalty at the level of individual rounds. SR and MT@4 evaluate the same target instructions with the same model and cumulative verifier, but differ in workspace origin and evaluation context: SR starts from a reference-completed state, whereas MT@4 depends on the agent-produced state accumulated across prior rounds. The comparison is therefore controlled for target instruction and verifier, and it is evidence that accumulated workspace state contributes substantially to the multi-turn decline.

Across all models and execution records, 57.0% of individual rounds that fail under MT@4 are solvable under SR from a reference-completed state (n=21,234 MT-failing rounds total, of which 12,111 pass under SR). The penalty grows with round depth: only 15.0% of round-1 MT failures are SR-solvable (n=886), rising to 55.6% at round 3 (n=1,957), 59.0% at round 7 (n=2,931), and above 80% beyond round 12 ($n \leq 584$). Figure 9 visualizes this progression.

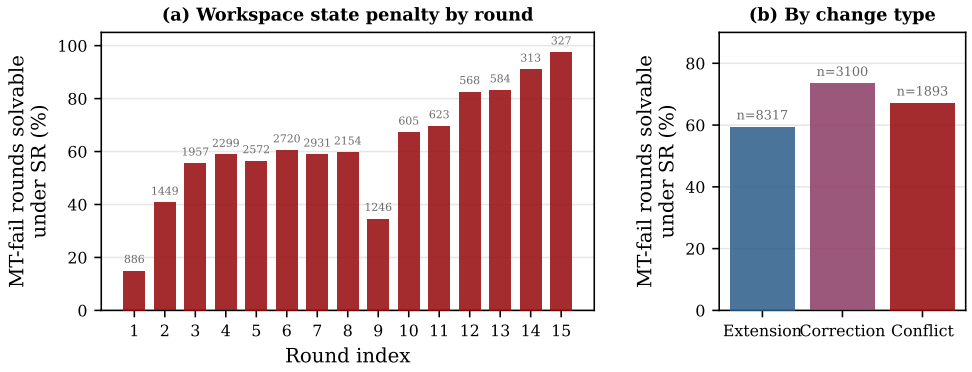

Figure 9: Workspace state penalty. (a) Percentage of MT-failing rounds that are solvable under SR, by round index. The penalty grows with depth: at later rounds, most failures involve target instructions that can be solved from a reference-completed state. Numbers above bars indicate the count of MT-failing rounds at that index. (b) The same metric grouped by change type; correction and conflict rounds show higher SR-solvability than extension rounds.

The penalty also varies by change type. Correction rounds show the highest fraction of SR-solvable MT failures (73.5%, n=3,100), followed by conflict (67.1%, n=1,893) and extension (59.4%, n=8,317). This ordering is consistent with the interpretation that requirement revision is particularly sensitive to workspace quality: correction and conflict rounds demand that the agent update previously implemented behavior, which is more likely to fail when the prior workspace already deviates from a reference-quality state. The extension–correction–conflict ordering should be interpreted with caution because change types are not randomly assigned to rounds and co-vary with round position and task content.

G.12 First-Failure-Round Distribution

Section 4.2 reports that lower-tier agents fail earlier than top-tier agents. This subsection presents the full distribution. For each trial, we record the round index at which the first failure occurs and normalize by the task's total number of rounds to produce a position in $[0, 1]$. Trials that pass all rounds are excluded.

Among lower-tier agents, 57.4% of trial failures occur in the first 20% of rounds (n=827 failing trials), consistent with basic requirement-satisfaction deficits. Mid-tier agents show 29.4% of failures in the first 20% (n=1,522), with a more uniform distribution across positions. Top-tier agents fail earliest in only 14.4% of cases (n=993), with failures spread more evenly and 19.1% occurring in the final 20% of rounds, reflecting the late-emerging regression and conflict-handling failures described in Section 4.3.

G.13 Token Consumption and Evaluation Cost

Evaluation cost. The full multi-round evaluation logged 3,657 execution records for 13 agents on 26 tasks and consumed approximately 24.1 billion input tokens and 333 million output tokens, or 24.4 billion total reported tokens. These records include resumed or fragmented Harbor executions rather than only the 13 $\times$ 26 $\times$ 4 top-level agent-task attempts. This total does not include single-round evaluation runs.

Within-round token consumption: passing vs. failing trials. To investigate whether failing trials exhibit different resource-use patterns, we compare agent output tokens of passing and failing trials at the same round index (controlling for the confound that later rounds naturally accumulate longer contexts). Across rounds 1–9, where both pass and fail samples are sufficient, failing trials produce 1.1–3.1$\times$ as many output tokens as passing trials at the same round (Figure 10). The ratio is lowest at round 1 (1.1$\times$; n_{pass}=2,760, n_{fail}=723) and highest at round 6 (3.1$\times$; n_{pass}=699, n_{fail}=54). We emphasize that the causal direction of this association is ambiguous: more difficult workspace states may simultaneously cause both failure and increased agent effort, and the pattern should be

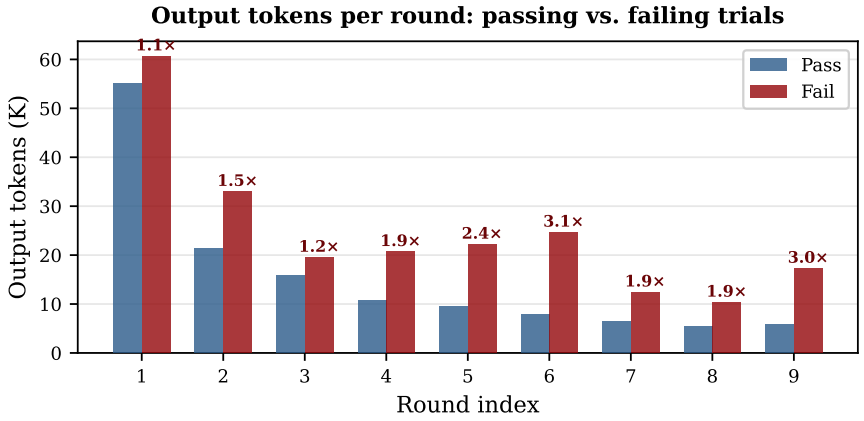

Figure 10: Output tokens per round for passing vs. failing trials (controlled for round index). Failing trials consistently produce more output tokens at the same round position. Numbers above fail bars indicate the fail-to-pass token ratio. Sample sizes decrease at later rounds due to fail-stop termination.

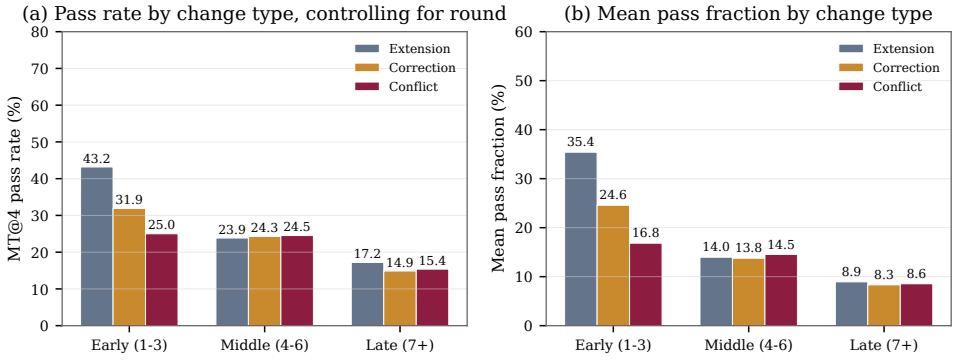

Figure 11: Change-type effect on pass rates, controlling for round position. (a) MT@4 pass rate by change type and round bucket. (b) Mean pass fraction (across four attempts) by change type and round bucket. Extension rounds show higher pass rates than correction and conflict rounds in the early bucket, where the signal is strongest; differences diminish in the middle and late buckets.

interpreted as a diagnostic observation rather than evidence that higher token production causes failure.

G.14 Change-Type Effect Controlling for Round Position

Figure 11 controls for round position by grouping rounds into early (1–3), middle (4–6), and late (7+) buckets, and comparing MT@4 pass rates across change types within each bucket. In early rounds, extension rounds show notably higher pass rates than correction or conflict rounds (43.2% vs. 31.9% vs. 25.0%). In middle rounds, the three change types converge (23.9% vs. 24.3% vs. 24.5%), and in late rounds extension again leads modestly (17.2% vs. 14.9% vs. 15.4%). The early-round difference is the strongest and cleanest signal, because round position is least confounded there. Mean pass fractions (panel b) show greater separation in the same direction for early rounds. These results support the interpretation that requirement revision (correction and conflict) imposes additional difficulty in early rounds, where the confound of round position is minimal.

H Responsible Release and Impact

Broader impact. EvoCode-Bench is intended to improve the measurement of coding agents under realistic iterative development conditions. The primary positive impact is more transparent evaluation: persistent workspaces, cumulative tests, and fail-stop scoring can reveal regression and

specification-tracking failures that are hidden by single-turn benchmarks. This may help researchers and practitioners deploy coding agents with more realistic expectations about long-horizon reliability.

The main negative impact is that higher-scoring coding agents can also be used to automate harmful software development. The benchmark does not include tasks for malware, credential theft, exploit development, or evasion, and released tasks focus on benign engineering workflows such as data processing, testing, migration, reporting, and configuration management. We nevertheless recommend treating benchmark improvements as capability measurements rather than deployment guarantees.

Assets and licensing. The released package includes task definitions (instructions, reference solutions, cumulative tests, and Docker environments), evaluation scripts, the Harbor multi-turn extensions, and machine-readable result summaries for all 13 evaluated models. The task format is documented in Appendix B, the Harbor evaluation protocol in Appendix F, and the model configuration in Appendix F.2. Existing infrastructure assets (Harbor [11] and the Terminus-2 harness [12]) are credited in the paper. The dataset release is marked CC-BY-NC 4.0, code and task packages include their corresponding license files, and users of closed-model APIs remain responsible for complying with the corresponding provider terms.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the benchmark contribution, persistent multi-turn execution setting, cumulative verification protocol, evaluation protocol, and main empirical findings.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Appendix A discusses limitations including benchmark scale, fail-stop underestimation, executable-test scope, and subjective design judgments. Appendix H discusses broader impact and responsible release.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: The paper does not include theoretical results.

   Guidelines:

   - The answer NA means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections 3.4 and 4.1 define the scoring protocol, agent-task matrix, attempts, single-round evaluation, and metrics. Appendix F describes the run artifacts needed to reproduce or audit reported scores.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The benchmark tasks, evaluation scripts, Harbor multi-turn extensions, and summary scripts are released through anonymized reviewer-accessible URLs. Appendix B, Appendix F, and Appendix H document the task format, run artifacts, license scope, and release policy.

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper does not train models. Sections 3.2, 3.4, and 4.1 specify task construction, Docker-based execution, model access, attempts, single-round fast-forward evaluation, and metrics.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper does not report confidence intervals or formal significance tests because the benchmark scale (26 tasks) limits the power of such tests. Instead, MT@4 aggregates four independent multi-round attempts per agent-task pair, SR provides a complementary reference-fast-forwarded view, and activity, style, and round-index breakdowns expose relevant sources of variation.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper does not fine-tune models and describes Docker-container execution, API-based model access, run artifacts, and multi-attempt evaluation in Sections 3.4 and Appendix F. It does not provide exact worker hardware, memory, or wall-clock time because those depend on model provider latency and task length.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: Yes, the research conducted in the paper fully conforms to the NeurIPS Code of Ethics in every respect.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix H discusses positive impacts from more realistic coding-agent evaluation and negative impacts from measuring capabilities that could improve harmful software automation.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: The paper does not release a high-risk model or scraped sensitive dataset. The released benchmark excludes malware, credential theft, exploit development, and evasion tasks; remaining responsible-release considerations are discussed in Appendix H.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Existing infrastructure assets such as Harbor and Terminus-2 are credited in the paper, and Appendix H states the release-license and provider-terms policy for benchmark code, tasks, and closed model APIs.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The paper introduces new benchmark tasks, cumulative tests, reference solutions, evaluation scripts, and result summarization scripts. Appendix B, Appendix F, and Appendix H document these assets, including the dataset license and release scope.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper does not involve crowdsourcing nor research with human subjects. Task curation, review, and annotation validation were performed by the authors or internal research staff as part of benchmark construction.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: The paper does not involve crowdsourcing nor research with human subjects; no participant study or external worker experiment was conducted.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLM-based coding agents are the subjects of evaluation. Section 3.2 describes LLM assistance in candidate task drafting and failure-analysis prompts, Appendix D.3 describes the LLM-assisted failure annotation protocol and human validation, and Sections 3.4 and 4.1 together with Appendix F.2 describe the evaluated model set, agent harness, and evaluation protocol.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2026/LLM`) for what should or should not be described.